

Technical Interview Preparation Guide

Web Development

JavaScript fundamentals, 50 React interview questions, system design frameworks, an algorithm study guide, and behavioral question prep.

1. JavaScript Fundamentals Guide

The concepts interviewers actually probe, and where candidates most commonly trip up on each.

Concept	Key Point	Common Gotcha
Closures	A function retains access to its enclosing scope's variables even after that scope has returned	Loop variables captured by reference with <code>var</code> , not value — all closures see the final value unless <code>let</code> or an IIFE is used
Scope & Hoisting	<code>var</code> is function-scoped and hoisted with an initial undefined; <code>let/const</code> are block-scoped and hoisted into a “temporal dead zone”	Referencing a <code>let/const</code> variable before its declaration throws, rather than silently returning undefined like <code>var</code>
this Binding	<code>this</code> is determined by how a function is called, not where it's defined (except arrow functions)	Arrow functions don't have their own <code>this</code> — they inherit it from the enclosing lexical scope, which is easy to forget in callbacks
Equality (<code>==</code> vs <code>===</code>)	<code>==</code> performs type coercion before comparing; <code>===</code> does not	Default to <code>===</code> always; <code>==</code> produces genuinely surprising results (e.g. <code>" == 0</code> is true)
Promises & Async/Await	<code>async/await</code> is syntactic sugar over promises, not a different concurrency model	Forgetting to <code>await</code> a promise inside a loop, or awaiting sequentially when the operations could run in parallel via <code>Promise.all</code>
Event Loop	JavaScript is single-threaded; the event loop processes the call stack, then microtasks (promises), then macrotasks (<code>setTimeout</code> , etc.)	Assuming <code>setTimeout(fn, 0)</code> runs immediately — it still waits for the current call stack and any pending microtasks to clear
Prototypal Inheritance	Objects inherit directly from other objects via the prototype chain, not from classes (class syntax is sugar over this)	Modifying a shared prototype method affects every instance, including ones already created
Array/Object Methods	<code>map/filter/reduce</code> are non-mutating; <code>push/splice/sort</code> mutate the original array	Assuming <code>Array.prototype.sort()</code> sorts numerically by default — it sorts as strings unless given a compare function

2. React Interview Questions (50)

Organized by category. Prepare the underlying concept, not a memorized answer — real interview questions will be phrased differently.

Core Concepts & JSX (Questions 1-8)

1. What is the virtual DOM, and how does React's reconciliation process use it?
2. What are the rules of JSX, and how does it compile down to JavaScript?
3. What's the difference between an element and a component in React?
4. Why does React require a unique key prop when rendering lists, and what happens if keys are missing or unstable?
5. What's the difference between controlled and uncontrolled components?
6. How does React's one-way data flow differ from two-way data binding?
7. What are React Fragments, and what problem do they solve?
8. What is the difference between props and state?

Component Lifecycle & Hooks (Questions 9-18)

9. Walk through what `useEffect` does and how its dependency array affects when it runs.
10. What's the difference between `useEffect` and `useLayoutEffect`?
11. How would you replicate `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount` using hooks?
12. What are the rules of hooks, and why do they exist?
13. When would you write a custom hook, and what makes a custom hook well-designed?
14. What problem does `useRef` solve that `useState` doesn't?
15. How does `useReducer` differ from `useState`, and when would you prefer it?
16. What is a stale closure in the context of hooks, and how does it happen?
17. How do you handle cleanup in `useEffect`, and why does it matter?
18. What's the difference between class component lifecycle methods and the mental model of hooks?

State Management (Questions 19-26)

19. When should state live in a parent component versus be lifted, versus live in context?
20. What problem does the Context API solve, and what are its performance limitations at scale?
21. What is prop drilling, and what are the trade-offs of the different ways to avoid it?
22. How would you decide between local component state and a global state management library?
23. What's the difference between derived state and stored state, and why does mixing them up cause bugs?
24. How do you avoid unnecessary re-renders caused by context value changes?
25. What are the trade-offs between colocating state near where it's used versus centralizing it?

26. How would you structure state for a form with many interdependent fields?

Performance Optimization (Questions 27-34)

- 27. What does React.memo do, and when does it fail to prevent a re-render?
- 28. What's the difference between useMemo and useCallback, and when is each actually necessary?
- 29. How does React batch state updates, and how has that changed across React versions?
- 30. What is code-splitting, and how do you implement it with React.lazy and Suspense?
- 31. How would you diagnose unnecessary re-renders in a real application?
- 32. What's the performance cost of creating new object or array literals as props on every render?
- 33. How does list virtualization improve performance, and when is it actually necessary?
- 34. What role does the key prop play in list performance, beyond just satisfying React's requirement?

Component Patterns & Architecture (Questions 35-42)

- 35. Compare higher-order components, render props, and custom hooks as patterns for sharing logic.
- 36. What is composition, and why does React favor it over inheritance?
- 37. What is an error boundary, and what categories of errors can it not catch?
- 38. How would you design a compound component (e.g. a custom Select with subcomponents)?
- 39. What's the difference between a presentational component and a container component, and is that distinction still useful with hooks?
- 40. How would you design a component API to be flexible without becoming overly complex?
- 41. What are portals, and what's a real use case where you'd need one?
- 42. How do you decide when a component has too many responsibilities and should be split?

Testing & Debugging (Questions 43-50)

- 43. What's the philosophy behind React Testing Library, and how does it differ from testing implementation details?
- 44. How would you test a component that fetches data asynchronously?
- 45. How do you test a custom hook in isolation?
- 46. What's the difference between a unit test, an integration test, and an end-to-end test in a React application?
- 47. How would you debug a component that's re-rendering far more often than expected?
- 48. How would you approach debugging a memory leak in a React application?
- 49. What tools would you use to profile a slow-rendering React application?
- 50. How do you approach writing tests for a component with complex conditional rendering logic?

3. System Design Frameworks

Front-end system design questions (“design a live comment feed,” “design an autocomplete component”) reward a structured approach more than a single “right” architecture. Use the same framework every time so you don't miss a dimension under interview pressure.

Step	What to Cover
1. Clarify Requirements	Functional requirements (what must it do), non-functional requirements (scale, latency, offline support), and explicit non-goals to bound the problem
2. High-Level Architecture	Component breakdown and data flow between them — sketch this before diving into any single piece in detail
3. State Management Approach	Where state lives, how it flows between components, and what triggers re-renders or re-fetches
4. Data Fetching Strategy	How and when data is fetched, cached, and invalidated; how loading and error states are represented
5. Performance Considerations	Rendering cost at scale (e.g. list virtualization), network efficiency (batching, pagination), and perceived performance (optimistic updates, skeleton states)
6. Edge Cases & Error Handling	Network failure, race conditions between fast-changing state and slow responses, and empty/loading/error states for every async operation

Practice Prompts

- Design an infinite-scroll social media feed
- Design a real-time notification system for a web application
- Design a type-ahead search/autocomplete component
- Design a collaborative, real-time text editor
- Design a client-side form builder with dynamic validation

4. Algorithm Study Guide

This is a study guide to patterns and approach, not a solutions manual — recognizing which pattern a problem fits is usually the actual skill being tested, more than memorizing any single solution.

Core Patterns to Know

Pattern	Recognize It By	Typical Use
Two Pointers	A sorted array or the need to compare elements from both ends	Finding pairs that sum to a target, reversing in place
Sliding Window	Finding a contiguous subarray/substring meeting some condition	Longest substring without repeating characters, maximum sum subarray of fixed size
Hash Map Lookup	Needing fast existence or count checks while iterating once	Two-sum, finding duplicates, frequency counting
BFS / DFS	Traversing a tree or graph structure	Shortest path in an unweighted graph (BFS), exploring all paths (DFS)
Dynamic Programming	A problem with overlapping subproblems and optimal substructure	Longest common subsequence, coin change, climbing stairs
Binary Search	A sorted (or monotonic) search space	Finding an element or boundary condition in logarithmic time

Interview Approach

- Clarify constraints and edge cases before writing any code — input size, duplicate values, empty input
- Say your approach out loud before coding it — interviewers weight your reasoning process heavily, not just the final code
- Start with a brute-force solution if the optimal approach doesn't come immediately, then explicitly discuss how to optimize it
- Test your solution against a normal case and at least one edge case before declaring it done

5. Behavioral Question Prep

Use the STAR structure (Situation, Task, Action, Result) — prepare 5-6 real stories from your own experience that can flex across multiple questions below, rather than one story per question.

Common Engineering Behavioral Questions

- Tell me about a time you disagreed with a technical decision made by a teammate or lead.
- Describe a production issue you debugged. How did you approach finding the root cause?
- Tell me about a time you had to learn a new technology or codebase quickly.
- Describe a time you had to push back on a deadline or scope you felt was unrealistic.
- Tell me about a piece of code or a system you're proud of. What made it good?
- Describe a time you received difficult feedback on your code or approach. How did you respond?
- Tell me about a time you had to make a trade-off between code quality and shipping speed.
- Describe a time you mentored or helped a less experienced teammate.

Worked Example

Question: “Tell me about a production issue you debugged.”

Situation: A checkout page started intermittently failing for a subset of users, with no clear pattern in initial error logs.

Task: I needed to identify the root cause quickly, since it was actively affecting revenue.

Action: I added structured logging around the failure point, correlated failures against browser and network conditions, and found the issue only occurred on slow connections where a race condition let a form submit before an async validation call resolved.

Result: I fixed the race condition and added a regression test for it specifically. The fix eliminated the failure pattern entirely, and the logging approach became the team's standard first step for intermittent production issues.